

TicketSphere

Regional Finale – Solution & Success Documentation

Application Maintenance Intelligent Ticket Triage Agent

Implementation-grounded documentation generated from the submitted
BrainByte_RegionalFinals project

1. Executive Summary

TicketSphere is an enterprise AI ticket intelligence platform implemented as a multi-agent ticket-triage workflow. It accepts synthetic/manual tickets and is structured for Jira-backed ingestion, masks PII/secrets, retrieves evidence from historical tickets and operational knowledge, classifies the issue, assesses priority, routes the ticket, performs self-critique and output verification, applies a human-approval gate, and supports ticket-system synchronization.

The implementation directly addresses the supplied problem statement: high-volume application maintenance tickets, inconsistent manual triage, weak severity-based routing, and the need for better resolution efficiency. The project also includes an enterprise console, RAG, guardrails, auditability, evaluation endpoints, feedback/HITL support, and Jira integration modules.

Item	Project implementation
Product	TicketSphere – enterprise AI ticket intelligence platform
Primary users	Support managers; Ops/Azure/AWS/GCP engineers; platform administrators
Backend	Flask, LangGraph, Pydantic, SQLAlchemy
AI/RAG	OpenAI-compatible LLM gateway/local Ollama fallback; Chroma; hybrid BM25 + dense retrieval; RRF
Data stores	SQLite for operational/governance data; Chroma for document/chunk content
Frontend	React 19, TypeScript, Vite, Ant Design, TanStack Query, Zustand, Recharts
Ticket source	Synthetic by default; Jira integration supported
Core workflow	normalize → enrich → grade → classify → assess → route → reflect → verify → gate → sync
Privacy	PII/secret detection, masking/redaction, input/output guardrails
Governance	RBAC/clearances, chunk-level ACL, hash-chained audit log, human approval

2. Problem Statement Alignment

The supplied challenge asks for a multi-agent system for ticket ingestion, NLP classification/prioritization, routing, manager visibility and override, two-way ticketing integration, and measurable improvements in classification accuracy, resolution time and routing precision.

Requirement	TicketSphere implementation	Evidence in project
Ticket ingestion	API/synthetic/Jira polling and shared ingest_and_triage	backend/api.py; backend/integrations/*;

	orchestration	backend/ai/agents.py
NLP classification	Dedicated classify node using retrieved context and structured LLM output	backend/ai/agents.py; backend/ai/prompts.py
Prioritization	Dedicated assess node plus deterministic priority rulebook/scoring controls	backend/ai/agents.py; docs/PRIORITY_RULEBOOK.md
Routing	Route node uses service/catalogue evidence with deterministic fallback	backend/ai/agents.py; backend/ai/tools.py
Historical data	Resolved-ticket precedent retrieval; gold labels are kept separate from indexed text	db/vectordb/data/seed/tickets; db/sqlite/models.py
Manager UI	Dashboard, triage queue, decision drawer, history, evals and control views	frontend/src/pages/*; frontend/src/components/*
Override/HITL	Human gate, ticket override APIs, feedback/review workflow	backend/api.py; db/sqlite/models.py
Two-way synchronization	Jira adapter, poller and write-back/tooling structure	backend/integrations/jira.py; backend/integrations/poller.py; backend/ai/tools.py
Explainability	Rationale, evidence/citations, confidence gates and trace/audit data	backend/ai/agents.py; backend/observability/*
Safety/privacy	Input injection checks, PII masking, output guardrails, role/ACL enforcement	backend/guardrails/*; backend/rag/anonymizer.py
Testing/evaluation	RAG evals plus held-out triage accuracy/routing/severity evaluation	backend/observability/ evals.py; /evals/run-triage

3. Solution Architecture

High-level flow:

Ticket Source (Synthetic / API / Jira)

|
v

Normalize + PII/Secret Masking + Input Guard

|
v

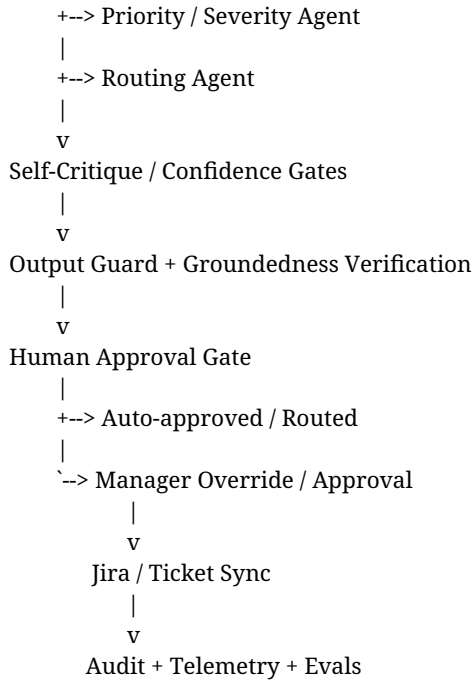
Hybrid RAG Enrichment

|– precedent tickets
|– runbooks
|– service catalogue
`– SLA policy

|
v

CRAG Grade / bounded re-retrieval

|
+--> Classification Agent
|



3.1 Agent roles

Agent / node	Responsibility	Key output
Normalize	Normalize ticket data, mask PII/secrets, run injection guard and feature extraction when required	Clean Ticket + guardrail state
Enrich	Retrieve evidence and detect likely duplicates	Retrieved evidence + duplicate candidate
Grade	CRAG-style relevance grading and one bounded query rewrite	Filtered/revised evidence
Classify	Categorize ticket and identify service/subcategory	Category, subcategory, service, rationale
Assess	Assess priority/severity against evidence and SLA context	P1–P4/priority, score, SLA target, rationale
Route	Select owning team from ticket/context/service evidence	Assigned team + routing rationale
Reflect	Self-critique evidence and confidence; may trigger one bounded enrichment retry	Issues, confidence adjustment
Verify	Run common output/groundedness guardrails	Rationale, groundedness, blocked status
Gate	Determine whether human approval is mandatory	needs_human + escalation reason
Sync	Assemble final decision, audit it and prepare status for synchronization	TriageDecision + status

4. Data Architecture

The project intentionally separates structured operational/governance data from unstructured knowledge content.

Store	Purpose	Examples
SQLite	Users, tickets, chat, feedback, evaluation results, audit trail, triage runs	users, tickets, chat_messages, feedback, eval_results, audit_log, triage_runs
Chroma	Chunked knowledge and ticket-history content with embeddings and metadata	runbooks, SLA policy, service catalogue, historical tickets
Seed corpus	Synthetic development/demo knowledge	10 seed tickets, 25 runbooks, 4 seed markdown documents
Held-out labels	Accuracy evaluation without leaking gold labels into retrieval text	true_category, true_severity, true_team, held_out

4.1 Privacy and data handling

- Synthetic ticket data is the project default; README explicitly states that the repository contains no real customer data.
- PII and secret detection is implemented in backend/guardrails/pii.py and masking/redaction is integrated into ticket normalization.
- Gold labels are stored separately from indexed chunk text to reduce evaluation leakage.
- Exports/logging can use irreversible redaction, while stable typed tokens preserve relationships during internal processing.
- Chunk-level ACL filtering is applied inside the vector query rather than as a post-filter.

5. Success Matrix

The challenge defines three primary success metrics: classification accuracy, average resolution-time reduction and routing precision. TicketSphere already contains direct measurement support for classification accuracy and routing precision against held-out gold labels. Resolution-time reduction is represented in the data model and historical metadata, but a controlled before/after benchmark is not populated by the repository itself; therefore it should be measured during the final demo rather than presented as an achieved number.

Success area	Metric	How TicketSphere measures it	Current evidence/status	Recommended target for finale
Classification	Classification accuracy	Held-out tickets: predicted category vs	Implemented evaluator; requires	≥90%

		true_category	generated/seeded held-out set for a numeric result	
Routing	Routing precision	Held-out tickets: predicted assigned_team vs true_team	Implemented evaluator; requires held-out run	≥90%
Priority	Severity/priority quality	Severity MAE + confusion matrix; priority is normalized P1–P4	Implemented evaluator	MAE ≤0.5 band; inspect confusion matrix
Resolution efficiency	Average resolution-time reduction	Compare baseline/manual triage time with AI-assisted triage time; historical resolution_minutes is available in corpus metadata	Measurement framework required; no achieved percentage should be claimed yet	≥30% reduction
Critical incidents	Critical-ticket recall	Add/derive P1 recall from gold labels and confusion matrix	Not directly exposed as a named KPI in current evaluator	≥95%
Explainability	Decisions with evidence/rationale	TriageDecision contains rationale and evidence citations	Implemented	100%
Human control	Human override/approval capability	Human gate, override flow and feedback tables/APIs	Implemented	100% of high-risk/low-confidence cases gated
Safety	Guardrail coverage	Input guard, PII, output guard, blocked reasons, audit events	Implemented	100% of ticket requests pass guardrail path
Integration	Two-way sync success	Jira adapter/poller/write-back path; sync attempts/errors stored on Ticket	Implemented path; requires connected Jira for live KPI	≥99% successful sync
AI quality	Groundedness / hallucination	RAG evals store groundedness, context precision/recall and hallucination	Implemented eval framework	≥0.80 groundedness; hallucination ≤0.20
Performance	Latency	Trace stages and stored message latency; triage trace captures stages	Telemetry implemented; final target should be benchmarked on demo hardware	Set p95 target after baseline
Reliability	Failed tickets not lost	run_triage and ingest_and_triage convert failures into human-	Implemented	100% of failures visible to humans

		review/failed states		
--	--	----------------------	--	--

5.1 Exact formulas for the three challenge KPIs

Classification Accuracy = correctly classified tickets / evaluated tickets

Routing Precision = correctly routed tickets / evaluated tickets

Resolution-Time Reduction % =

(manual baseline average time - AI-assisted average time)

/ manual baseline average time * 100

Important: target percentages in this document are recommended evaluation targets, not achieved results. The actual achieved values must come from the project evaluator/demo run.

6. Evaluation & Test Plan

The repository contains two evaluation families.

Evaluation	Purpose	Implementation
RAG/assistant evaluation	Groundedness, context precision, context recall, hallucination and latency/token telemetry	backend/observability/evals.py; POST /evals/run
Triage evaluation	Classification accuracy, routing precision, severity MAE and confusion matrix against held-out gold labels	backend/observability/evals.py; POST /evals/run-triage
Retrieval A/B	Compare vector vs hybrid retrieval on exact IDs, error codes, symptoms and policy/catalogue queries	run_retrieval_ab() in backend/observability/evals.py
Audit integrity	Verify hash chain of audit events	GET /audit/verify
Human feedback	Capture positive/negative feedback and corrected answers	/feedback APIs + Feedback model

6.1 Evaluation dataset currently defined in code

- The RAG evaluation set contains 12 representative cases, including SLA, ownership, runbook, historical-ticket and refusal scenarios.
- The retrieval evaluation set contains 12 probes covering exact IDs, error codes, symptoms, policy and catalogue queries.
- The triage evaluator supports up to 100 held-out cases by default and notes that a generated held-out set should be created if none exists.

6.2 Required final test execution

1. Seed/reset the vector database and ensure the intended synthetic corpus is indexed.
2. Generate or load the held-out gold-labeled ticket set.
3. Run the triage evaluator with rerun=True so the current graph is actually exercised.
4. Record classification accuracy, routing precision, severity MAE and confusion matrix.
5. Run the RAG evaluation and record groundedness, context precision/recall, hallucination and latency.
6. Run positive/negative/ambiguous/high-risk live scenarios and verify the human gate.
7. Run a Jira-connected synchronization scenario if credentials/environment are available.
8. Measure manual baseline triage time and AI-assisted triage time using the same scenario set.

7. Regional Finale Demo Scenarios

Scenario	Expected AI behavior	What to show judges
P1 production outage	Classify application/service, assign Highest/P1, identify critical evidence, gate for approval if required	Priority rationale, citations, confidence and manager approval
Payment/API 5xx spike	Retrieve matching runbook + precedent, classify and route to owning team	Evidence-first triage and suggested first action
Cloud database failure	Use error code/application clues and service catalogue	Hybrid retrieval advantage for identifiers
Low-risk access/password issue	Classify as lower priority and route to correct support function	Contrast with critical case
Ambiguous/low-evidence ticket	Lower confidence and send to human gate	Safe uncertainty handling
Prompt-injection/secret-containing ticket	Block or sanitize malicious/untrusted content; audit the event	Responsible AI guardrails
Manager override	Human changes team/priority and records reason	Reversibility and human control
Jira sync	Write approved decision/status back to ticket system	Two-way enterprise integration

8. User Experience & Interface

The frontend is an enterprise operations console rather than a generic chatbot. The frontend specification defines support-manager and engineer workflows and uses Ant Design with server-side tables and TanStack Query/Zustand separation.

Screen	Purpose
--------	---------

Login	Role-based access entry
Dashboard	Manager-level operational overview and KPI visualization
Triage / Queue	View incoming/open tickets, priorities, routing and status
Decision Drawer	Inspect AI rationale, evidence, confidence and approve/override
History	Review past decisions and outcomes
Evals	Run/view AI and triage evaluation results
Documents	Manage/index knowledge sources
Control	Governance/administration controls
Chat	Ask grounded questions over the ticket/knowledge corpus

8.1 UX success criteria

- AI decisions should be understandable without reading logs: category, priority, team, confidence, rationale and evidence are visible.
- High-risk/uncertain cases require human intervention rather than silent automation.
- Severity/status should not rely on color alone; labels are part of the frontend contract.
- The workflow is designed around minimal clicks to accept/override a decision.

9. API & Integration Surface

The backend API layer is Flask-based and uses a consistent response envelope: successful responses use data/meta; failures use error/code/message.

Area	Representative endpoints / capability
Ticketing	Ticket ingestion, ticket listing/detail, approval/override, confidence recalculation
Analytics	GET /analytics/triage; GET /analytics/usage; GET /analytics/traces; GET /analytics/messages
Evaluation	POST /evals/run; POST /evals/run-triage; GET /evals
Feedback	POST /feedback; GET /feedback; PATCH /feedback/<id>/review
Governance	GET /audit; GET /audit/verify
Integrations	Jira poll/sync/webhook modules; synthetic source fallback

The exact endpoint list should be kept synchronized with backend/api.py before submission; this document intentionally summarizes the implementation rather than duplicating every route.

10. Security, Governance & Responsible AI

Control	Implementation
Authentication	JWT-based authentication in Flask API
Authorization	Roles: admin, manager, engineer, viewer; engineer clearances include ops/azure/aws/gcp
Knowledge access	ACL metadata is enforced inside retrieval
PII/secrets	Detection, stable masking and irreversible redaction paths
Prompt injection	Input guard before ticket reasoning; blocked tickets go to human gate
Output safety	Schema validation, banned-claim checks, output guard and groundedness checks
Auditability	Append-only, hash-chained audit log with verification endpoint
Human oversight	Highest/uncertain/low-confidence/guardrail/duplicate cases can require approval
Secrets	Configuration is environment-driven; .env.example is provided and actual credentials should not be committed
Failure handling	Triage failures become visible failed/human-review states rather than disappearing

11. Observability & Traceability

- Trace stages cover major triage operations such as enrichment, generation/decision and verification.
- Chat messages persist latency, prompt tokens, completion tokens and groundedness.
- Triage runs and audit events retain decision metadata, confidence gates and guardrail counts.
- Analytics endpoints expose usage, traces and message telemetry to the frontend.
- Evaluation results are persisted for comparison across runs.

12. Project Structure Relevant to the Solution

backend/
 ai/ LLM, prompts, tools, LangGraph agents
 rag/ indexing, retrieval, chunking, embeddings, anonymization
 guardrails/ input/output/PII/governance controls
 observability/ traces and evaluation
 integrations/ Jira, polling, synthetic source, SLA notifications
 chatbot/ chat sessions, memory and context
 api.py Flask API routes
db/
 sqlite/ operational relational store
 vectordb/ Chroma store + seed corpus
frontend/
 src/pages/ Dashboard, Triage, Queue, History, Evals, Documents, Control, Chat

src/components/ Ticket table, decision drawer, chatbot, charts, shared UI
docs/
FLOW.md request/graph flow
JUDGES_QA.md technical rationale and trade-offs
PRIORITY_RULEBOOK.md deterministic priority rubric
.claude/plans/
BLUEPRINT.md architecture/build contract
INTEGRATION.md integration planning
rag.md RAG plan

13. Documentation Required for Final Submission

Document	Required content	Status based on repository
README / Quickstart	Problem, setup, run commands, demo accounts, reset/seed	Present
Architecture document	System components, data flow, agent graph, stores, integrations	Present across BLUEPRINT/FLOW/README; consolidate for judges
Multi-agent design	Agent roles, handoffs, retries, reflection, human gate	Present in BLUEPRINT and agents.py
RAG design	Chunking, embeddings, retrieval, ACL, hybrid fusion, reranking	Present in RAG modules/plans; consolidate
AI/model document	Model tiers, fallback, prompts, rationale, token/latency trade-offs	Partially distributed across config/llm/prompts/plans; recommended to consolidate
Data dictionary	Ticket fields, gold labels, document metadata, users/roles	Present in models.py and seed metadata; recommended as standalone
Evaluation report	Actual accuracy, routing precision, severity MAE, RAG metrics, test cases	Framework present; actual final numbers must be generated
Success metrics report	Three challenge KPIs + formulas + baseline/after measurements	Framework documented here; resolution-time baseline must be measured
API documentation	Endpoint, auth, request/response examples, error cases	API exists; standalone OpenAPI/Postman-style artifact recommended
Security/privacy	PII, secrets, RBAC, ACL, audit, retention, prompt injection	Implementation present; consolidate
Live demo script	Scenario inputs, expected outputs, judge talking points	Recommended; scenarios included in this document
User guide	Manager/engineer workflows, approve/override, queue usage	Frontend exists; add screenshot-based guide
Deployment/runbook	Prerequisites, env vars, database/vector seed, startup and troubleshooting	README has quickstart; expand for final handoff
Limitations/future scope	Known gaps, benchmark limits, production-readiness next steps	Recommended

14. Evidence Gaps to Close Before the Finale

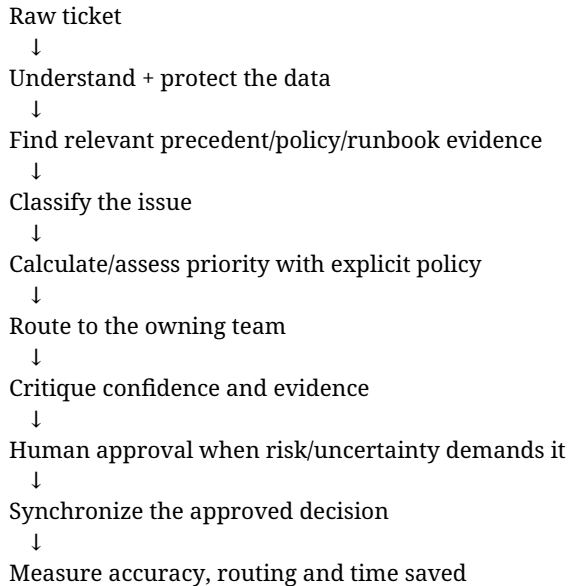
- Run the held-out triage evaluator and insert the actual classification accuracy and routing precision into the final presentation/report.
- Run a controlled resolution-time experiment. Use a fixed scenario set and record manual triage time versus AI-assisted triage time; do not claim a reduction without this baseline.
- Generate/seed the held-out evaluation dataset if it is not present. The evaluator itself explicitly reports when no gold-labeled held-out tickets exist.
- Capture one real Jira two-way synchronization test if the environment permits; otherwise clearly label the demo as simulated/synthetic integration.
- Export the final evaluation result and confusion matrix as an appendix artifact.
- Add screenshots of Dashboard, Queue, Decision Drawer, Evals and Override flow to the final user guide.
- Verify that demo credentials/secrets are safe and that no actual API tokens are committed.
- State clearly which metrics are measured, which are target thresholds, and which are not yet measured.

15. Limitations & Prototype-to-Enterprise Readiness

Area	Current prototype posture	Next enterprise step
Data volume	Designed for moderate daily ticket influx and local Chroma/SQLite	Move to managed relational/vector stores and load-test at expected enterprise volume
Model gateway	OpenAI-compatible provider plus local Ollama fallback	Formal model gateway, model registry and versioned evaluation gates
Jira	Integration module exists; custom field IDs are environment/site specific	Complete site-specific field mapping and integration contract testing
Metrics	Evaluation code exists	Automate CI benchmark thresholds and historical trend reporting
Observability	Local trace/telemetry persistence	Centralized metrics/logs/traces and alerting
Secrets/config	Environment-driven	Managed secret store and environment-specific configuration
Knowledge governance	ACL and document governance metadata exist	Formal approval/versioning/retention workflow for KB changes
Human feedback	Feedback and corrected-answer storage exist	Closed-loop review process and retraining/evaluation pipeline

16. Judge-Facing Value Proposition

The strongest story for the finale is not simply that an LLM classifies tickets. TicketSphere turns triage into an evidence-backed, auditable and reversible operational decision.



This directly maps to the challenge's business outcomes: less manual triage, more consistent classification and routing, faster handling of urgent incidents, and stronger operational trust through evidence, confidence, auditability and human control.

Appendix A – Final KPI Capture Sheet

KPI	Baseline	AI result	Formula/result	Evidence file/run
Classification accuracy	_____	_____	correct / total	_____
Routing precision	_____	_____	correct / total	_____
Severity MAE	N/A	_____	mean absolute band error	_____
Average manual triage time	_____ min	N/A	baseline	_____
Average AI-assisted triage time	N/A	_____ min	baseline vs AI	_____
Resolution/triage time reduction	_____	_____	(baseline-AI)/baseline	_____
Groundedness	N/A	_____	evaluation score	_____
Context precision	N/A	_____	evaluation score	_____
Context recall	N/A	_____	evaluation score	_____
Hallucination rate	N/A	_____	1-groundedness	_____
Jira sync success	N/A	_____	successful / attempts	_____

Appendix B – Demo Evidence Checklist

- ☐ Architecture diagram visible in deck
- ☐ Live ticket submitted
- ☐ Multi-agent trace/flow shown
- ☐ Classification + priority + routing shown
- ☐ Evidence citations/rationale shown
- ☐ Confidence shown
- ☐ Human approval/override demonstrated
- ☐ Guardrail/refusal scenario demonstrated
- ☐ Jira/ticket synchronization demonstrated or clearly simulated
- ☐ Evaluation result shown with actual numbers
- ☐ Resolution-time comparison shown
- ☐ Documentation repository/package available to judges